

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 3 – AI News Briefing Service

Team:	The Three Musketeers	Date:	September 19, 2026
Repo:	https://github.com/aliassefoglul/Final_Project_01/	Tag:	v1.0-final
Members:	Ali Huseynli (@aliassefoglul), Alish Hajiyev (@alishhajiyev), Elsen Samadzada (@elshansamadzada7)		

Executive Summary

We built **NewsBrief**, a service that collects articles from several news sources, removes duplicate stories, summarizes the remaining articles, and creates a personalized Markdown digest for each user. The system uses six configured sources, PostgreSQL for persistent data and caching, and the course-provided AI package for summarization and topic labeling. For a benchmark of 20 fetch operations, the concurrent implementation finished in 3.00 seconds, compared with 13.79 seconds for the sequential version, which corresponds to a $4.6\times$ speedup. We also tested the system against a real failure during development: the Reuters HTML source returned HTTP 401 responses, but the failure was isolated to that source and the other sources continued normally. Another important lesson was that database design required more work than we expected, especially when we changed the user's topic list from JSONB to normalized relational tables. If we continued the project, we would add source-health tracking, a shared provider-rate-limit mechanism, and an integration test using the real PostgreSQL container.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	3
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	4
3.3 Rate limit & politeness	5
4 Robustness & Error Handling	5
4.1 Wrapping the AI module	5
4.2 Failure-mode analysis (one concrete incident)	5

4.3	Input validation	6
5	Storage & Caching	6
5.1	Schema	6
5.2	Caching policy	7
6	Testing	7
6.1	Strategy & coverage	7
6.2	Notable tests	8
7	Deployment & Reproducibility	8
7.1	Docker image	8
7.2	Configuration	8
8	Limitations & Future Work	9
9	Tools & Acknowledgements	9
	References	10
A	Appendix — Reproducing the benchmark	10

1. Project Overview

1.1 Problem framing & scope

The main purpose of the system is to collect recent news for a user, remove repeated coverage of the same story, summarize the remaining articles, and produce a short personalized briefing. Articles are collected from RSS feeds and one HTML-scraped source. After fetching, the system performs URL/content normalization and near-duplicate detection before sending new content to the AI service. The final digest is stored as a Markdown file under `digests/`.

There are two user-facing entry points. The main one is the CLI command `python -m src.cli run-daily -user <name>`. We also implemented a small FastAPI web application in `src/api.py`, which provides signup, login, topic preferences, and access to the latest digest. The web interface was added as an extension to the required CLI workflow so that the project could also be demonstrated through a browser.

The configured LLM provider is Google Gemini, using `LLM_PROVIDER=google` and the model `gemini-3.6-flash`. We do not use the embedding provider. The two required deduplication stages are sufficient for the current implementation, so adding embeddings would not provide a necessary function in the pipeline.

We also chose not to add semantic clustering based on embeddings. The specification provides embeddings as an available AI primitive, but the required URL/content and near-duplicate stages already give us the duplicate filtering needed for this project. Instead, we spent the available development time on the fetch layer, database design, testing, and the additional web interface.

1.2 Provider choice and the AI module contract

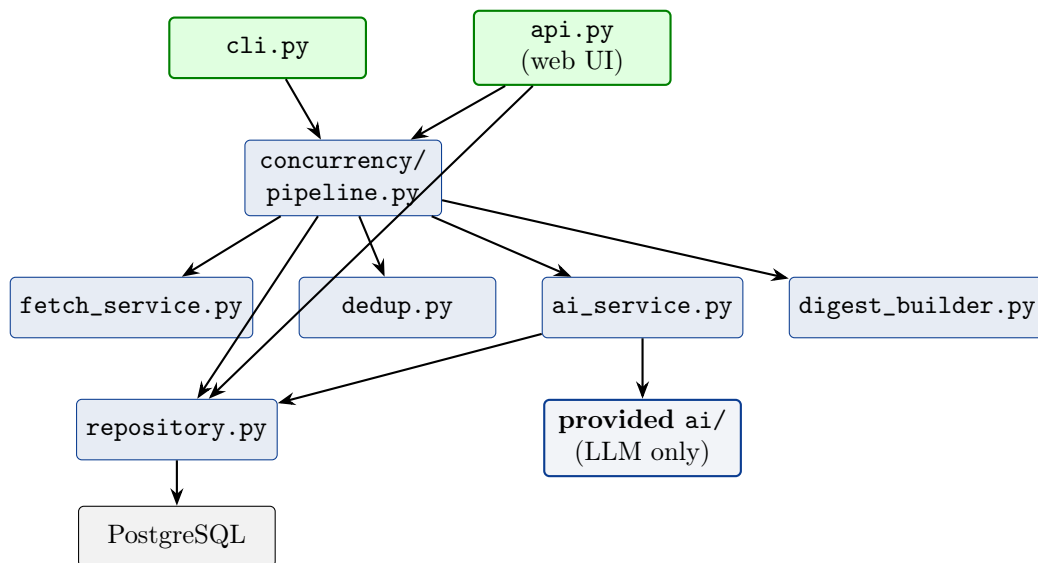
We use Google Gemini through the provider factory supplied by the course package. The configured model is `gemini-3.6-flash`. No embedding model is used. Our code calls `ai.summarize_and_label` from `src/services/ai_service.py`; this is the only direct call from our service layer into the provided AI package.

We did not modify any file inside `ai/`. The application-specific code is located under `src/`, while the supplied package remains unchanged.

The provided AI package validates the returned data against its `LabeledSummary` schema. This means that invalid enum values or other schema violations are rejected before a result is returned to our service layer. Provider errors are handled separately by `AIService`, which retries transient provider failures. When the retry limit is reached, the pipeline records the failure for that article and continues with the other articles.

2. Architecture

2.1 Module map



The two entry points are `cli.py` and `api.py`. Both use the pipeline in `concurrency/pipeline.py`. The pipeline coordinates fetching, deduplication, AI processing, digest creation, and persistence.

`services/fetch_service.py` is responsible for retrieving articles. `core/dedup.py` performs normalization and duplicate detection. `services/ai_service.py` is the only application module that directly communicates with the supplied `ai/` package. `core/digest_builder.py` creates the final Markdown output. Database access is kept in `storage/repository.py`, while PostgreSQL contains the persistent user, preference, account, and processed-content data.

2.2 OOP design & key abstractions

```

1 class NewsSource(abc.ABC):
2     """Common interface for configured news sources."""
3     name: str
4
5     @abc.abstractmethod
6     async def fetch(
7         self, client: httpx.AsyncClient
8     ) -> list[Article]:
9         raise NotImplementedError
10
11
12 @dataclass
13 class RssSource(NewsSource):
14     feed_url: str
15     name: str = ""
16

```

```

17     async def fetch(
18         self, client: httpx.AsyncClient
19     ) -> list[Article]:
20         ...

```

We use `NewsSource` as an abstract base class because RSS and HTML sources have different fetching implementations but expose the same operation to the rest of the application. The pipeline therefore does not need to know which concrete source it is using. The common contract also makes it straightforward to add another source without changing the orchestration code.

We use composition in other parts of the system. For example, `AIService` and `Deduplicator` are separate components used by the pipeline rather than subclasses of a common base class. There is no useful type hierarchy between them, so composition keeps the responsibilities separate.

2.3 Data flow across module boundaries

Our application models in `src/models.py` include `Article`, `LabeledSummary`, `Source`, `ProcessedArticle`, `BriefingItem`, `User`, and `Account`. These models define the data passed between the main application components.

We do not use unstructured dictionaries as the normal interface between modules. The exception is the internal `DigestItem` helper in `core/digest_builder.py`, which is only used while constructing the final digest and is never stored or sent across an external service boundary.

The models supplied by `ai/` are used only around the AI call. Our own `Article` model contains application-specific fields such as `id`, `canonical_url`, and `content_hash`, which are needed by fetching, deduplication, and storage. The conversion between the two model sets is kept in `ai_service.py`.

3. Concurrency & Performance

3.1 Why this concurrency model

The fetch layer uses `asyncio` together with `httpx.AsyncClient`. Fetching articles is mainly an I/O-bound task because the application spends most of its time waiting for remote servers to respond. Using asynchronous requests allows several sources to be processed while other requests are waiting.

The maximum number of concurrent fetches is controlled by `MAX_PARALLEL_FETCHES`, which is set to 8 by default. This value follows the project specification and is large enough for the configured sources without creating an unnecessarily large number of simultaneous requests.

A thread pool could also handle network requests, but it would introduce thread management without providing a clear advantage for this workload. With `asyncio`, the concurrency is explicit in the source-fetching code and works naturally with the existing asynchronous HTTP client.

When the semaphore is set to 1, requests are effectively processed one at a time, so the implementation approaches the sequential case. Increasing the value above the number of independent source tasks does not provide additional benefit. For the current six-source configuration, a limit of 100 therefore does not mean that 100 requests will be executed at once.

3.2 Sequential-vs-concurrent benchmark

Workload	<i>N</i>	Sequential (s)	Concurrent (s)	Speedup
fetch (<code>scripts/bench.py</code>)	20	13.79	3.00	4.6×

The benchmark used 20 fetch operations and a maximum concurrency of 8. The sequential version took 13.79 seconds, while the concurrent version took 3.00 seconds. This gives a measured speedup of approximately $4.6\times$.

The result is lower than an ideal $8\times$ improvement because the requests do not all take the same amount of time and because the six configured sources limit the amount of useful source-level parallelism. The Reuters source also failed during the benchmark and went through its retry logic, which added additional waiting time.

3.3 Rate limit & politeness

The fetch layer uses `tenacity` to retry temporary HTTP and transport failures. Each request can be attempted up to three times with exponential backoff. Fetch concurrency is separately limited by `Semaphore(MAX_PARALLEL_FETCHES=8)`.

The AI layer has its own retry policy. Provider errors can be retried up to four times with exponential backoff, with the delay capped at 20 seconds. These two mechanisms are separate because HTTP fetching and LLM requests have different failure conditions.

During development, we observed real Gemini quota errors. The free-tier configuration returned HTTP 429 responses after the request limits were reached. The logged error included `RESOURCE_EXHAUSTED` and indicated the daily free-tier request quota. Once the daily quota was exhausted, retrying could not restore the request capacity, so the affected articles were skipped and the pipeline continued.

To reduce unnecessary requests during development, we added `AI_DRY_RUN` and `MAX_ARTICLES_PER_RUN`. These settings allowed the rest of the pipeline to be tested without repeatedly consuming the available LLM quota.

4. Robustness & Error Handling

4.1 Wrapping the AI module

`AIService` in `src/services/ai_service.py` is the single application-level wrapper around `ai.summarize_and_label`. Provider errors are retried four times using exponential backoff, with a maximum delay of 20 seconds. Other exceptions are not automatically retried because they may represent deterministic application errors rather than temporary provider problems.

The supplied AI function is synchronous, so our service executes it through `asyncio.to_thread` and places a 30-second timeout around the operation. This prevents one slow provider call from blocking the rest of the asynchronous pipeline indefinitely.

The service logs the article title and a short prefix of the content hash. Retry attempts are logged at warning level. The full article content is not written to the logs.

Before the AI call, the article must already have a content hash produced by the deduplication stage. The returned object is validated by the supplied AI package. Successfully validated summaries are then stored in the database and can be reused on later runs.

4.2 Failure-mode analysis (one concrete incident)

The most concrete failure observed during the benchmark was the HTML-scraping source returning HTTP 401 responses from Reuters. This occurred during the real `scripts/bench.py -N 20` benchmark rather than through an artificial test injection.

The fetch function retried the request three times. After the retries were exhausted, the exception was caught at the source level. The source was logged as failed and returned no articles, while the

remaining RSS sources continued processing.

From the user's perspective, the run still completed and produced its output. The failed source did not terminate the complete pipeline. The log contained the HTTP 401 error and the message indicating that the source was skipped after the retry attempts.

The incident also exposed a limitation. The application does not currently distinguish between a temporary source failure and a source that has been unavailable for a long period. As a result, the same source can consume retry attempts on every future run until its URL or configuration is changed.

4.3 Input validation

The CLI requires the `-user` argument through `argparse`. The current implementation does not apply an additional username-format restriction.

The web interface checks that usernames are not empty, passwords contain at least eight characters, and password confirmation matches the original password. Login failures use the same generic error message for an unknown username and an incorrect password.

Fetches RSS entries without a title or link are ignored. HTML pages are also skipped when the required title or article text cannot be extracted. URLs and article content are normalized before they enter the deduplication process. Valid data is then represented using the application's typed models.

One current limitation is in the topic-preference form. The submitted values are converted directly to `TopicEnum`; an invalid topic value submitted outside the normal browser form can therefore result in a `ValueError`. This should be converted into controlled validation handling before a production deployment.

5. Storage & Caching

5.1 Schema

```
CREATE TABLE users (
    user_id          TEXT PRIMARY KEY,
    preferred_length TEXT NOT NULL DEFAULT 'medium'
                        CHECK (preferred_length IN ('short','medium','long'))
);

CREATE TABLE user_preferred_topics (
    user_id TEXT NOT NULL
        REFERENCES users(user_id) ON DELETE CASCADE,
    topic   TEXT NOT NULL
        CHECK (topic IN ('Politics','Tech','Sports',
                        'Business','Health','Science',
                        'World','Entertainment','Other')),
    PRIMARY KEY (user_id, topic)
);

CREATE TABLE user_excluded_sources (
    user_id TEXT NOT NULL
        REFERENCES users(user_id) ON DELETE CASCADE,
    source  TEXT NOT NULL,
    PRIMARY KEY (user_id, source)
);

CREATE TABLE accounts (
    user_id          TEXT PRIMARY KEY
        REFERENCES users(user_id) ON DELETE CASCADE,
    password_hash    TEXT NOT NULL,
```

```

    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE processed_urls (
  content_hash   CHAR(64) PRIMARY KEY,
  canonical_url  TEXT NOT NULL UNIQUE,
  summary        TEXT,
  topic          TEXT,
  sentiment      TEXT,
  processed_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

The database uses separate tables for user topics and excluded sources instead of storing lists inside the `users` table. This keeps individual values atomic and makes the schema easier to query. Foreign keys with `ON DELETE CASCADE` also ensure that related preference and account records are removed when the corresponding user is deleted.

User information and preferences are source-of-truth data. The `processed_urls` table stores derived AI results and is used as the application’s persistent summary cache.

User updates are performed inside an explicit database transaction. This means that changes affecting several related tables are either committed together or rolled back together.

5.2 Caching policy

The application caches the AI result for an article using its SHA-256 content hash. The result is stored in `processed_urls`. When an article with the same content hash is encountered again, the existing result can be used without another LLM request.

There is no time-based cache expiration. The reason is that the cache key is based on the normalized article content rather than only the URL. If the content changes, the hash changes and the application treats it as new content.

The main case where an old result could become outdated is a change in the AI prompt or model. A summary generated by one model would remain associated with the same content hash even after a model change. A production version could include a model or prompt version in the cache key.

A numerical cache-hit percentage is not included because the current project evidence does not contain a reproducible hit/miss count from one complete demo run. The existing logs distinguish cache hits from fresh AI calls, so the percentage can be measured directly from a controlled two-run test.

6. Testing

6.1 Strategy & coverage

The application tests do not call the real LLM provider. The pipeline tests use a fake `AIService` and a fake repository, while asynchronous operations are controlled with `AsyncMock` and `monkeypatch`. HTTP requests are intercepted with `respx`, so the fetch tests do not depend on external web-sites.

The full test suite contains 69 passing tests in the Docker environment. The supplied AI smoke tests also pass.

Suite	Coverage
src/ (our SE layer only)	86.7%
Provided ai/ smoke tests	passing (15/15)

The coverage for the application code is 86.7%. The combined coverage reported by `pytest-cov` is approximately 90%, but the more useful number for the project is the coverage of `src/`, since this is the code written by the team.

For type checking, `mypy -explicit-package-bases -disable-error-code=import-untyped src` reports no errors in the application code. The remaining reported errors are inside the supplied AI provider files and were not changed because the `ai/` package is read-only for this project.

6.2 Notable tests

Happy path. `test_pipeline.py::test_run_daily_pipeline_happy_path` runs the main pipeline with controlled fetch, AI, and repository components. It verifies that the expected article appears in the generated digest and that a duplicate containing URL tracking parameters is removed.

Concurrency. `test_fetch.py::test_fetch_all_sources_runs_sources_concurrently` creates three sources and makes one of them raise `httpx.ConnectError`. The test checks that the other sources still return their articles. This verifies that a single failed source does not terminate the complete fetch operation.

Error path. `test_pipeline.py::test_run_daily_pipeline_skips_articles_that_fail_ai_labeling` forces AI labeling to fail and checks that the pipeline still finishes and writes an appropriate digest. This test is important because an LLM failure should affect the corresponding article rather than terminate the complete daily run.

The main missing test is a real integration test against PostgreSQL. The current storage tests mock `asyncpg`, which keeps them fast but means that a mismatch between the SQL schema and an actual database could still be missed by unit tests.

7. Deployment & Reproducibility

7.1 Docker image

The project is run through Docker Compose. The main build command is:

```
docker compose up -d --build
```

The application consists of the PostgreSQL database, the scheduler/application container, and the FastAPI web service. A single pipeline run can be started with:

```
docker compose run --rm app \
  python -m src.cli run-daily --user khagani
```

The Dockerfile uses `python:3.12-slim` and a two-stage build. Build dependencies are installed in the builder stage and the resulting Python packages are copied into the runtime stage. This keeps the compiler toolchain out of the final image.

The exact image size was not recorded in the project evidence used for this report, so no specific size is claimed here. It can be obtained from the final build with:

```
docker images
```

7.2 Configuration

The main configuration values are stored in `.env` and documented through `.env.example`.

- `LLM_PROVIDER`: selects the LLM provider.
- `LLM_MODEL`: selects the configured model.

- The corresponding provider API key: required when the application makes an actual LLM request.
- `DATABASE_URL`: specifies the PostgreSQL connection. A reachable database is required for normal application operation.
- `HTML_SCRAPE_URL`: specifies the HTML source.
- `HTML_SCRAPE_SOURCE_NAME`: gives the configured name of that source.
- `SESSION_SECRET_KEY`: used for web sessions. A development default exists, but a real deployment should provide a private value.
- `DEDUP_NEAR_DUPLICATE_THRESHOLD`: controls the near-duplicate threshold.
- `MAX_PARALLEL_FETCHES`: controls fetch concurrency and defaults to 8.
- `FETCH_TIMEOUT_SECONDS`: controls HTTP request timeouts.
- `LOG_LEVEL`: controls logging verbosity.
- `DIGESTS_DIR`: specifies where generated digests are written.

Most of the operational settings have defaults, while the database and LLM configuration are required for the corresponding parts of the application to work.

8. Limitations & Future Work

- **HTML scraping is source-dependent.** The `HtmlScrapeSource` depends on the current HTML structure of its target website. A change in the page structure can cause the source to return no usable articles until its selectors are updated.
- **There is no secondary LLM provider.** If the configured provider remains unavailable after the retry attempts, affected articles are skipped. The application does not currently switch automatically to another provider.
- **Topic validation needs stronger handling.** A manually forged HTTP request containing an invalid topic value can currently result in a `ValueError`. This should be converted into a controlled validation response.
- **Password hashing could be stronger for production.** The current implementation uses PBKDF2-HMAC-SHA256. It is suitable for the scope of the coursework and does not require another dependency, but a production authentication system could use a password-hashing library designed specifically for this purpose.
- **There is no shared provider budget across runs.** The fetch semaphore limits concurrency inside a run, but separate scheduled runs do not share a common LLM request budget. This can become important when multiple users are processed close together.

With another week, we would first add a real PostgreSQL integration test and improve validation in the web interface. We would then add source-health tracking so permanently unavailable sources are not retried indefinitely. A shared provider budget would also make scheduled multi-user runs safer with respect to external API limits.

9. Tools & Acknowledgements

During development, the team used AI-based tools as supplementary development and writing aids. The project requirements, architecture, implementation choices, testing process, and final technical

decisions were reviewed by the team and tested against the actual application.

Claude was used during development to help review parts of the code structure and suggest improvements to selected implementation details. The suggestions were checked against the project requirements and adapted where necessary.

ChatGPT was also used for language support, including improving grammar, organizing explanations, and reviewing parts of the report and presentation text. The final report describes the implementation and test results of the project itself, and the team reviewed the final technical content before submission.

References

The project references were official documentation for Python `asyncio`, HTTPX, `tenacity`, `Pydantic`, `FastAPI`, `asyncpg`, PostgreSQL, Docker Compose, and the Gemini API which was used when implementing and checking framework-specific functionality. Additionally, the Topic 3 specification and the course-provided `ai/` package was used for this project.

- [1] `tenacity` documentation: <https://tenacity.readthedocs.io/>
- [2] `asyncpg` documentation: <https://magicstack.github.io/asyncpg/current/>
- [3] `FastAPI` documentation: <https://fastapi.tiangolo.com/>
- [4] `pydantic-settings` documentation: https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [5] PostgreSQL documentation, transactions: <https://www.postgresql.org/docs/current/tutorial-transactions.html>
- [6] Python `asyncio` documentation: <https://docs.python.org/3/library/asyncio.html>
- [7] HTTPX documentation: <https://www.python-httpx.org/>
- [8] `Pydantic` documentation: <https://docs.pydantic.dev/>
- [9] Docker Compose documentation: <https://docs.docker.com/compose/>
- [10] Gemini API documentation: <https://ai.google.dev/gemini-api/docs>
- [11] AI-ENG-110 Topic 3 specification (TOPIC.md), AI Academy, National AI Center, Spring 2026.

A. Appendix — Reproducing the benchmark

The benchmark can be reproduced from the repository with the following commands:

```
git clone <repo>
cd <repo>

cp .env.example .env
# Configure the required provider settings.

docker compose build app

docker compose run --rm app \
  python scripts/bench.py --N 20
```

The benchmark compares the sequential and concurrent fetch implementations using the same workload. The reported project result was 13.79 seconds for the sequential run and 3.00 seconds for the concurrent run.

**End of report — thank you for reading.*